

Análise e Projeto de Sistemas

Introdução à Unified Modeling Language

UML

UML – Unified Modeling Language

- ❑ É uma linguagem visual para modelar sistemas Orientados a Objetos, que foi aprovada em 1997 pela OMG (Object Management Group). Atualmente, a UML está na versão 2.5;
- ❑ Ela poderá ser empregada para a visualização, a especificação, a construção e a documentação de artefatos que façam uso de sistemas complexos de software. Cada elemento gráfico possui uma:
 - Sintaxe: forma predeterminada de desenhar o elemento;
 - Semântica: O que significa o elemento e com que objetivo deve ser usado;

A sintaxe e a semântica são extensíveis;

A UML é uma linguagem de modelagem para:

- ☐ Visualização
- ☐ Especificação
- ☐ Construção
- ☐ Documentação
- ☐ Comunicação



**Elementos
Estruturais**

**Elementos
Comportamentais**

**Elementos de
Agrupamento**

**Elementos de
Anotação**

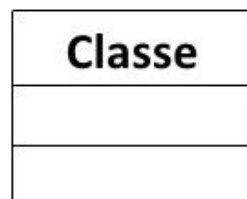
São as partes estáticas de um modelo, representando elementos que são ou conceituais ou físicos.

Exemplos:

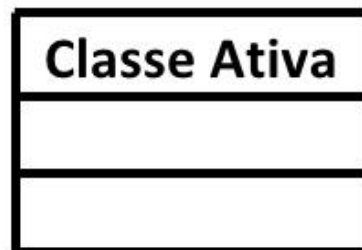
- ☐ Classe
- ☐ Interface
- ☐ Use Cases
- ☐ Componente
- ☐ Nó



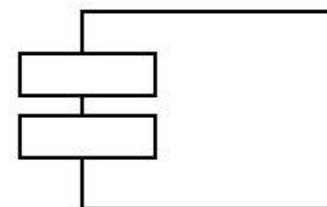
ITENS ESTRUTURAIS



Classe



Classe Ativa



Componentes



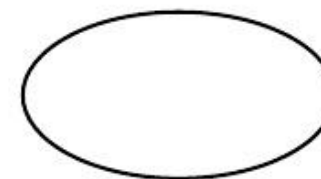
Nós



Interface



Colaborações



Caso de Uso

São as partes dinâmicas dos modelos da UML.

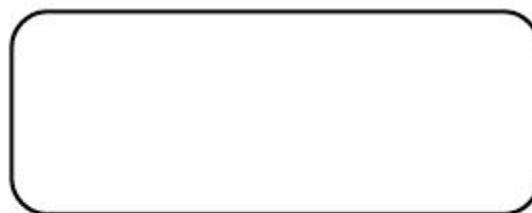


ITENS COMPORTAMENTAIS

❖ **Interação:** Comportamento que abrange um conjunto de mensagens trocadas entre objetos num contexto específico



❖ **Máquina de estados:** Especifica as seqüências de estados pelas quais objetos e interações passam durante sua existência em resposta a eventos.



São partes organizacionais dos modelos da UML.

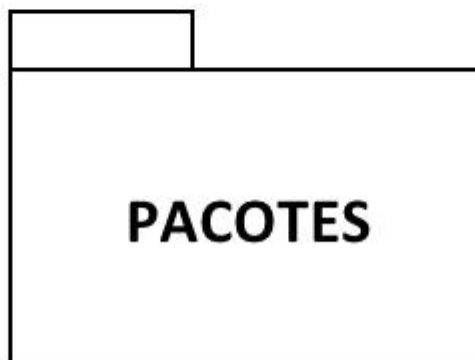


ITENS EM UML



Itens de Agrupamento

São partes organizacionais dos modelos de UML.
Servem para a organização de elementos (como
itens estruturais ou comportamentais) em
grupos.



- ❑ **Pacotes** - mecanismo para organização de elementos dentro de grupos

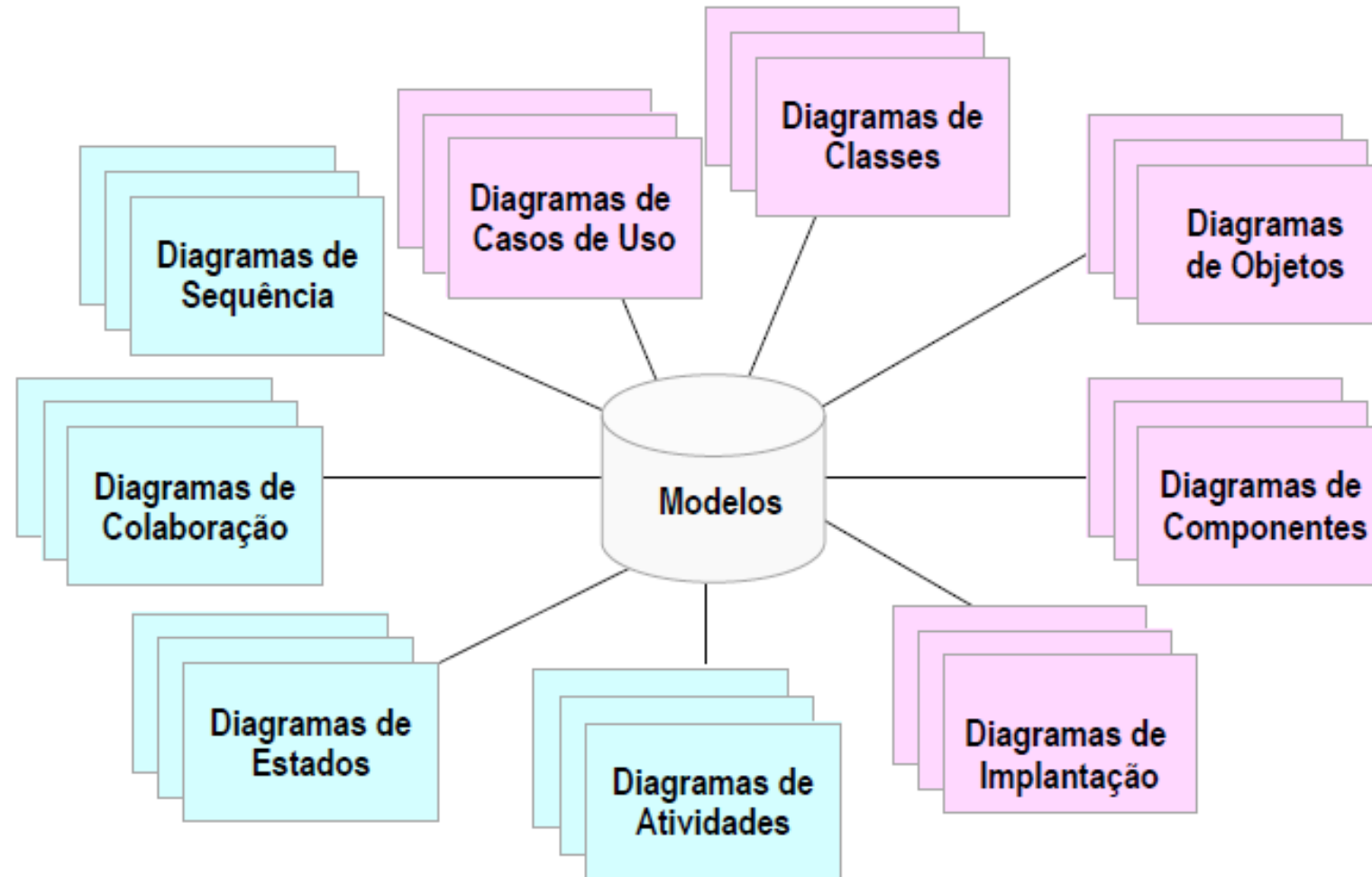


São partes explicativas dos modelos da UML. São comentários que você aplica para descrever, iluminar e remarcar elementos no modelo.

Exemplo:

- ❑ Nota - símbolo contendo restrições ou comentários que são melhor expressadas em textos

São representações gráficas de um conjunto de elementos. São desenhados para visualizar um sistema de diferentes perspectivas.



Servem facilitam o entendimento de um sistema mostrando a sua “visão externa”

- ☐ São usados para modelar o contexto de um sistema, subsistema ou classe
- ☐ São uma das maneiras mais comuns de documentar os requisitos do sistema
- ☐ Delimitam o Sistema
- ☐ Definem a funcionalidade do sistema

- São especialmente importantes na organização e modelagem das principais funcionalidades de um sistema. Use Case é a especificação de sequências de ações atender a uma funcionalidade do sistema, interagindo com seus agentes



Um use case é uma unidade funcional que descreve o comportamento de um elemento da aplicação;

Contém sequências de ações, interagindo com os atores que usam a aplicação;

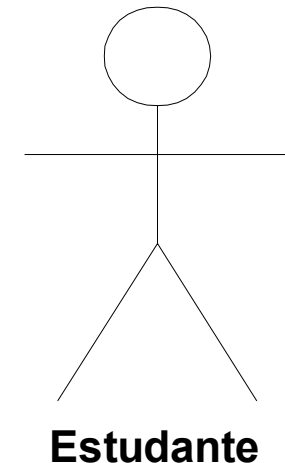
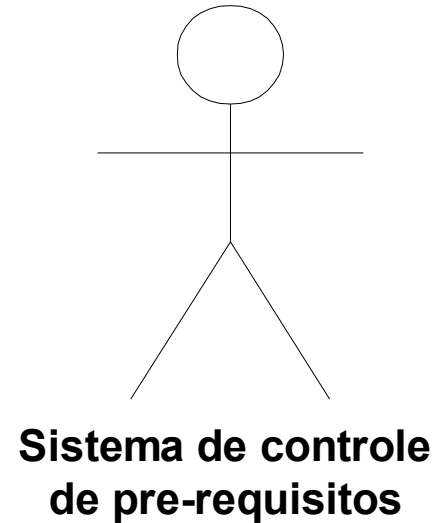
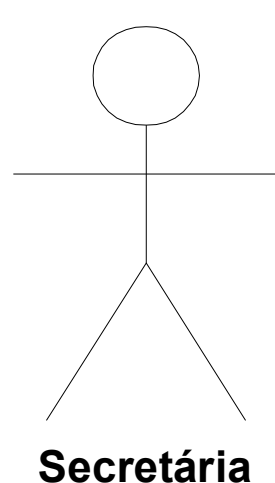
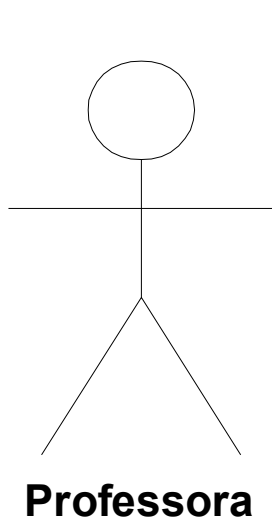
Inclui variantes, rotinas de erro, etc. que o sistema executa para produzir um resultado observável para um ator.

Matricular aluno

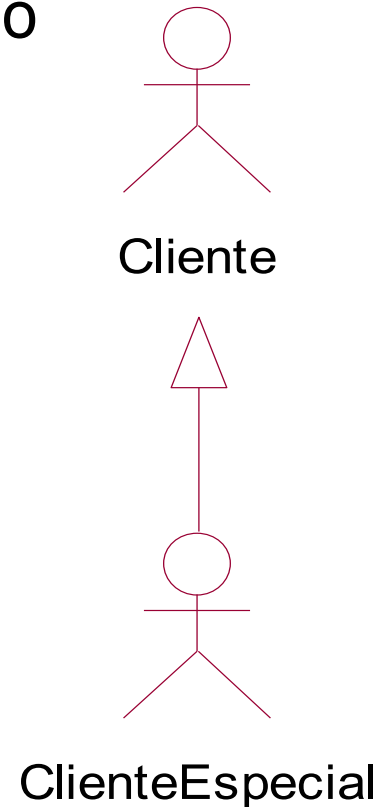
**Solicitar
histórico**

**Verificar
pré-requisitos**

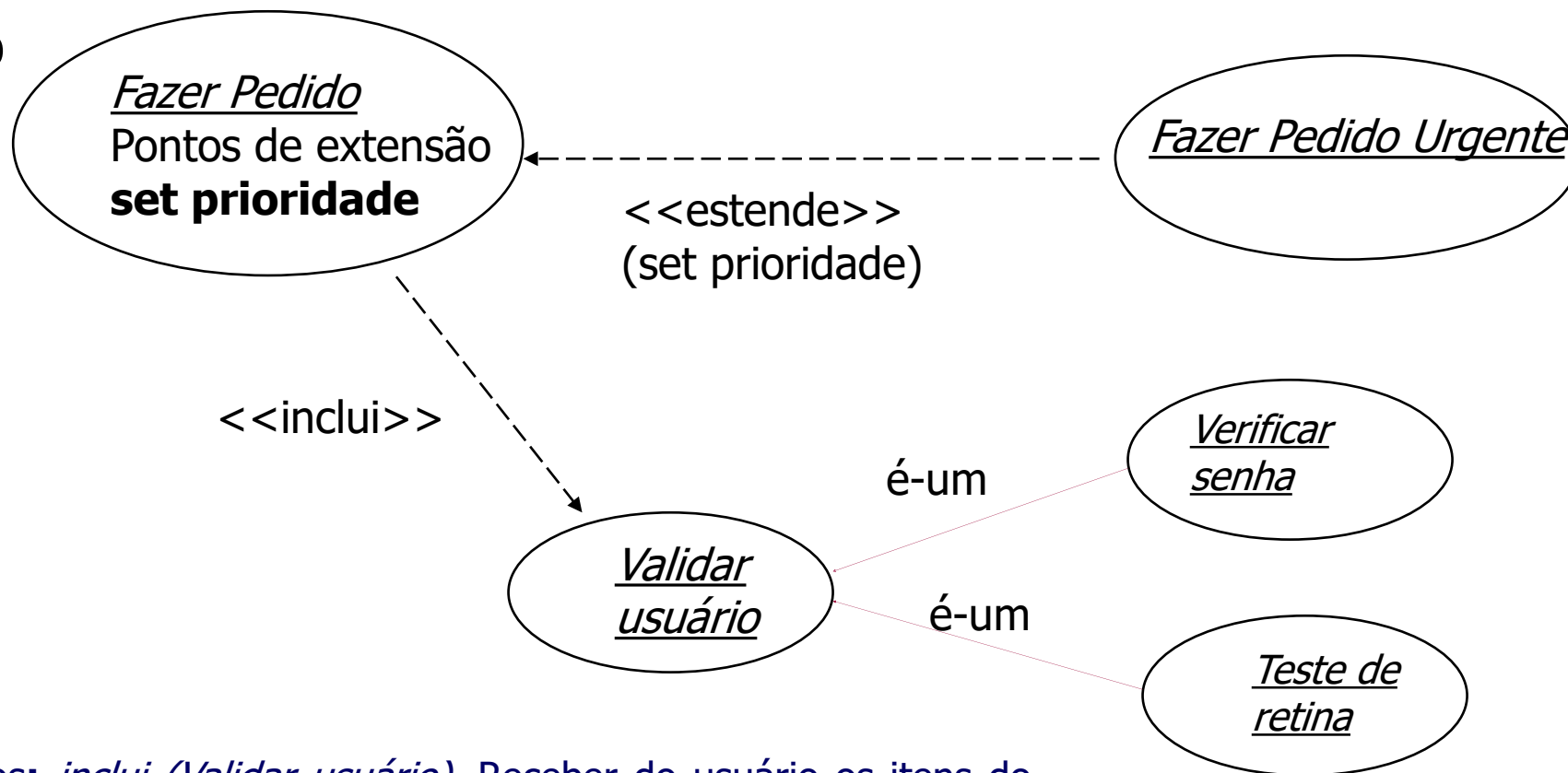
- O sistema será descrito através de vários use cases que são executados por um número de atores
- Atores constituem as entidades do ambiente do sistema
- São pessoas ou outros subsistemas que interagem com o sistema em desenvolvimento



- É possível definir tipos gerais de atores e especializá-los usando o relacionamento de especialização



- Generalização
- Inclusão
- Extensão



Use Case Fazer Pedido

Fluxo principal de eventos: *inclui* (Validar usuário). Receber do usuário os itens do pedido. (set prioridade). Submeter o pedido para processamento

- **Classes** são o elemento mais importante de qualquer sistema orientado a objetos
- Uma classe é uma descrição de um conjunto de objetos com os mesmos **atributos, relacionamentos, operações e semântica**
- Classes são usadas para capturar o **vocabulário** de um sistema
- Classes são abstrações de elementos do domínio do problema, como "Cliente", "Banco", "Conta"

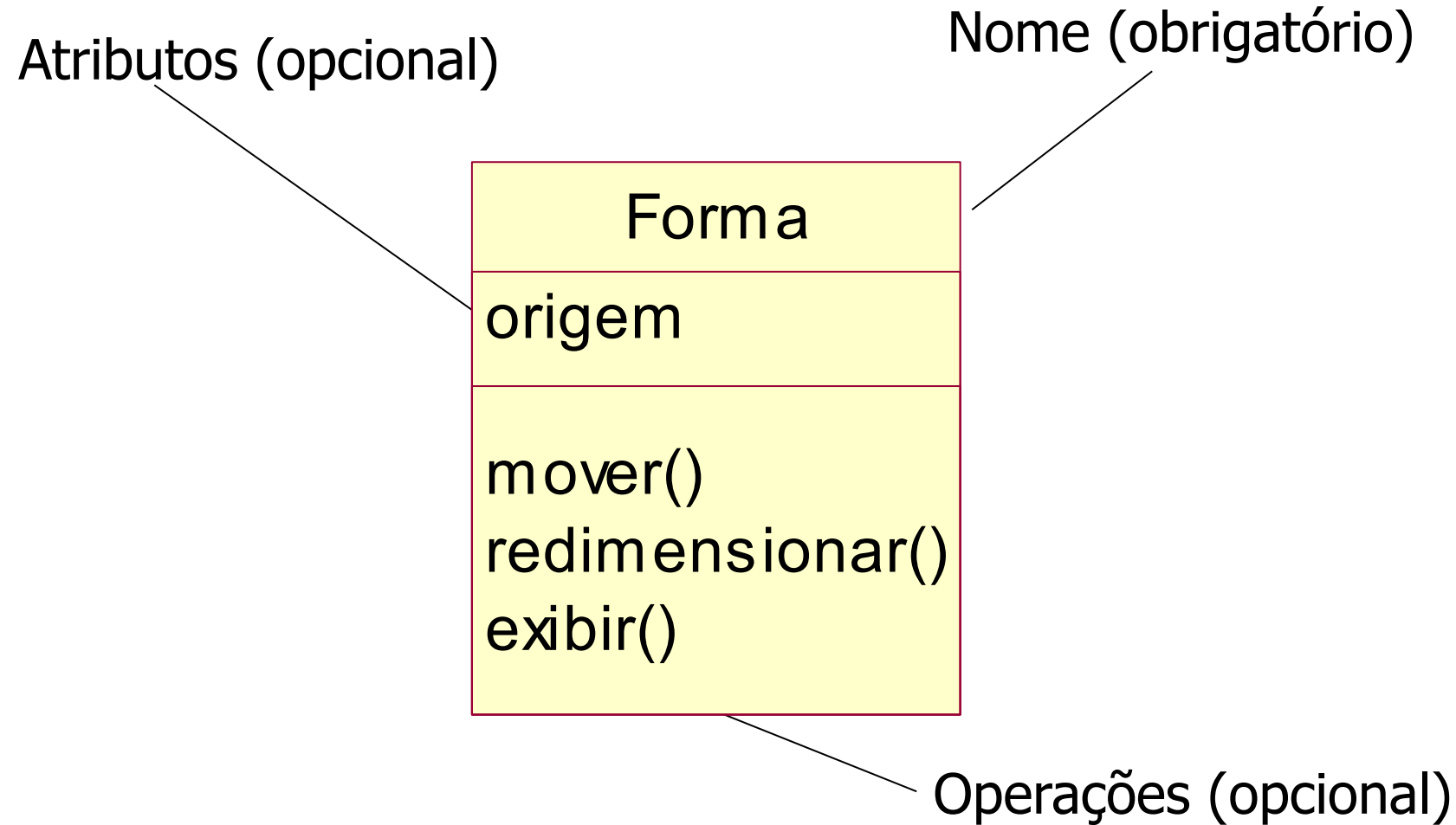
- Toda classe deve ter um nome que a distinga das outras classes
- Um nome pode ser simples (apenas o nome), ou pode ser precedido pelo nome do pacote em que a classe está contida

Conta

Banco

Cliente

Exceções::ClienteNãoCadastrado



- Um atributo representa alguma propriedade do que está sendo modelado, que é compartilhada por todos os objetos da classe
- Os atributos descrevem os dados contidos nas instâncias de uma classe
- Em um momento dado, um objeto de uma classe conterá valores para todos os atributos descritos na sua classe

- Atributos podem ser identificados apenas com nomes

Cliente

nome
endereço
telefone

- Atributos podem ter seus tipos (ou classes) especificados e terem valores padrão definidos

Parede

altura : real
largura : real
espessura : real
viga : boolean = false

- Uma operação é uma abstração de alguma coisa que se pode fazer com um objeto e que é compartilhada por todos os objetos da classe
- Um classe pode ter qualquer número de operações, inclusive nenhuma
- Operações são o meio de alterar os valores dos atributos

Retângulo

mover()
aumentar()
diminuir()

- Como para os atributos, você pode especificar uma operação apenas com seu nome
- Você pode também especificar a assinatura da operação: seus parâmetros, o tipo desses parâmetros e o tipo de retorno

- Você pode usar marcações de acesso para especificar o tipo de acesso permitido aos atributos e operações
 - Classificador pode ser classes, interfaces, componentes, nós, use cases, subsistemas
- + público: todos os classificadores podem usar
- # protegido: qualquer descendente do classificador poderá usar
- privado: somente o próprio classificador poderá usar

- Poucas classes vivem sozinhas
- Tipos de relacionamentos especialmente importantes na modelagem orientada a objetos:
 - **Associações**
 - **Agregação**
 - **Composição**
 - **Dependências**
 - **Generalizações**
 - **Realização**

Os relacionamentos ligam as classes/objetos entre si criando relações lógicas entre estas entidades. Os relacionamentos podem ser dos seguintes tipos:

- ❑ **Associação** - especifica que objetos de um elemento (classe) estão conectados a objetos de outros elementos;



- ❑ **Agregação** - relacionamento fraco do tipo "é parte de". É um tipo especial de associação

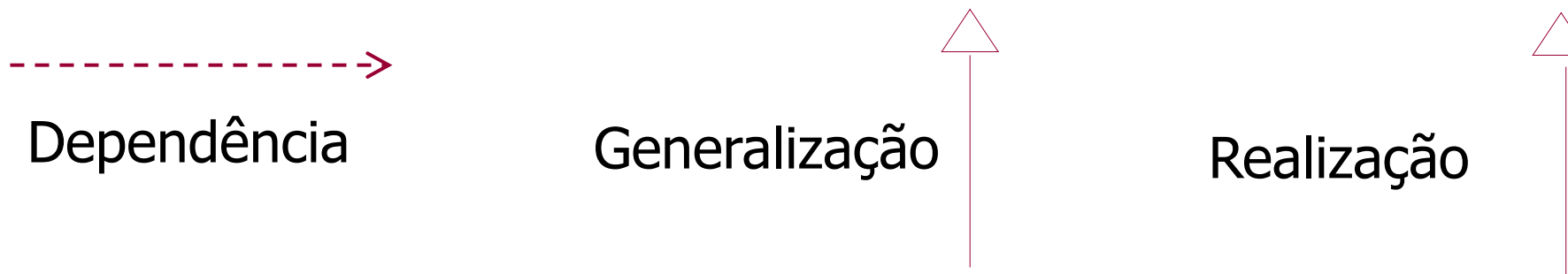
Agregação

Composição



- ❑ **Composição** - relacionamento forte do tipo "é parte de". A composição entre um elemento (o "todo") e outros elementos ("as partes") indica que as partes só existem em função do "todo".

- **Dependência** - relacionamento de uso, no qual uma mudança na especificação de um elemento pode alterar a especificação do elemento dependente
- **Generalização (herança)** - relacionamento entre descrições mais gerais e descrições mais específicas, com mais detalhes sobre alguns dos elementos gerais
- **Realização** - relacionamento entre uma interface e o elemento que a implementa

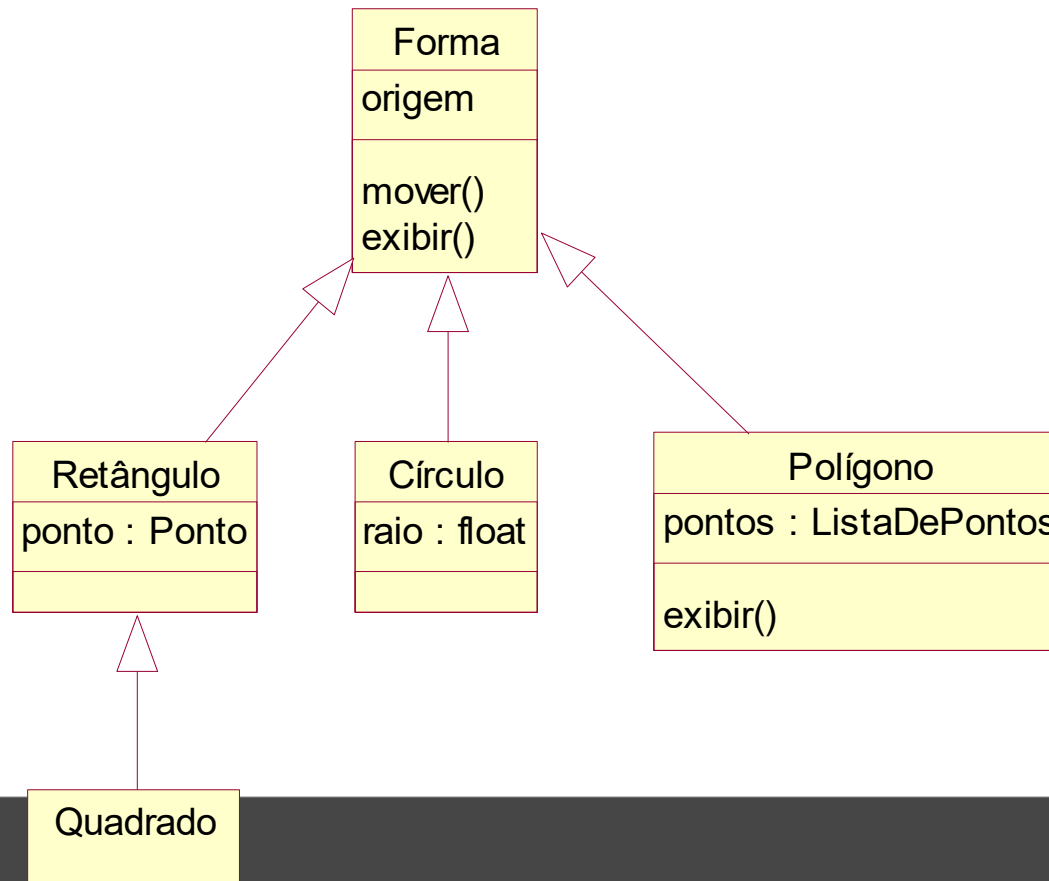


- Dependências são relações de **uso**
- Uma dependência indica que mudanças em um elemento (o "servidor") podem afetar outro elemento (o "cliente")
- Uma dependência entre classes indica que os objetos de uma classe **usam** serviços dos objetos de outra classe

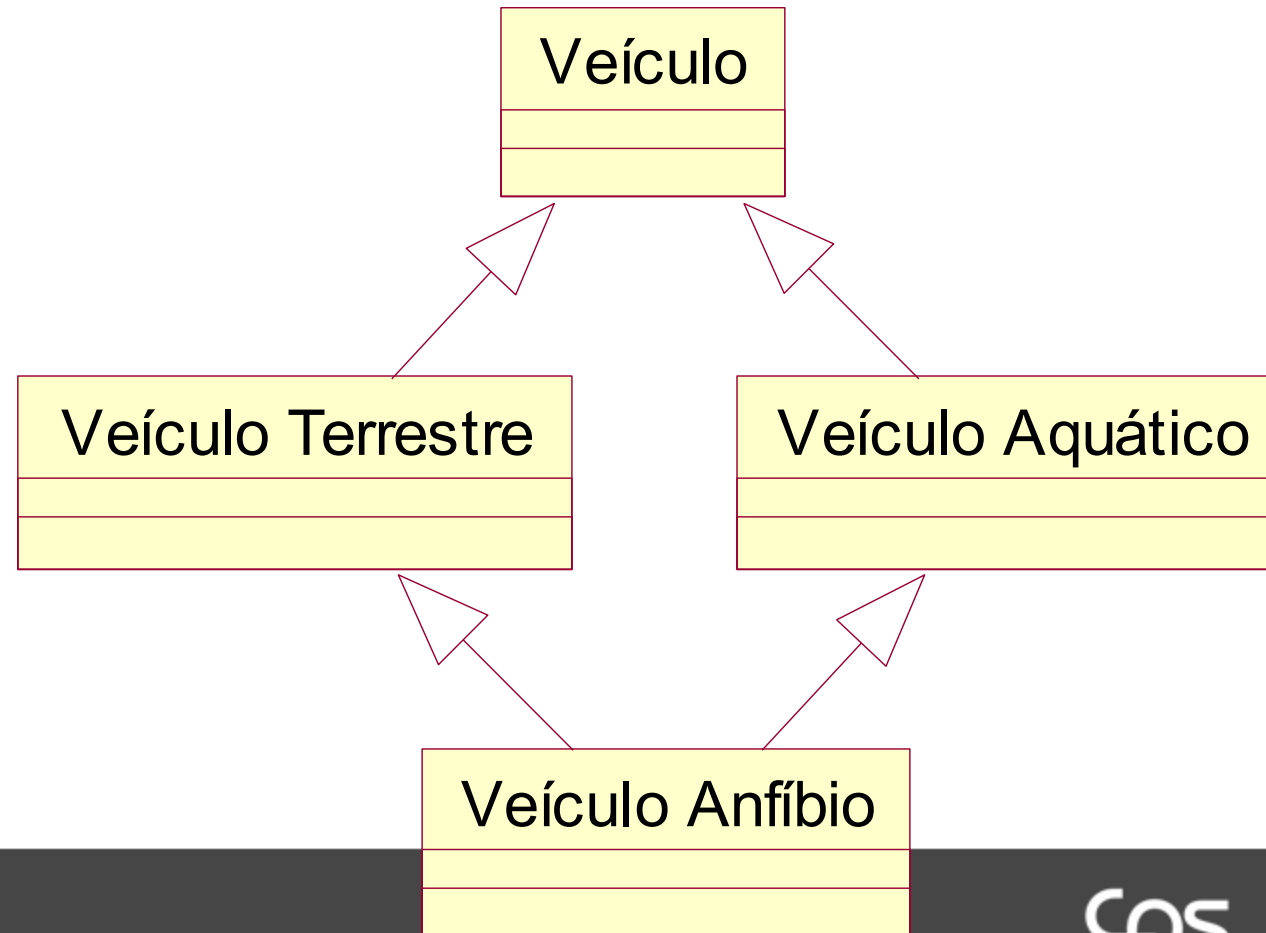


- derive – a fonte é computada a partir do destino
 - friend – a fonte tem acesso privilegiado ao destino
 - instanceof – o objeto fonte é instância da classe destino
 - powertype – o destino é composto por subconjuntos da fonte
 - derivado – a fonte é computada a partir do destino
-
- Entre pacotes: access e import
 - Entre use-cases: extend e include
 - Entre objetos: become, call e copy

- Relacionamento entre um elemento mais geral (chamado de superclasse ou pai) e um mais específico (chamado de subclasse ou filho)



- Ocorrem múltiplas superclasses para uma mesma subclasse



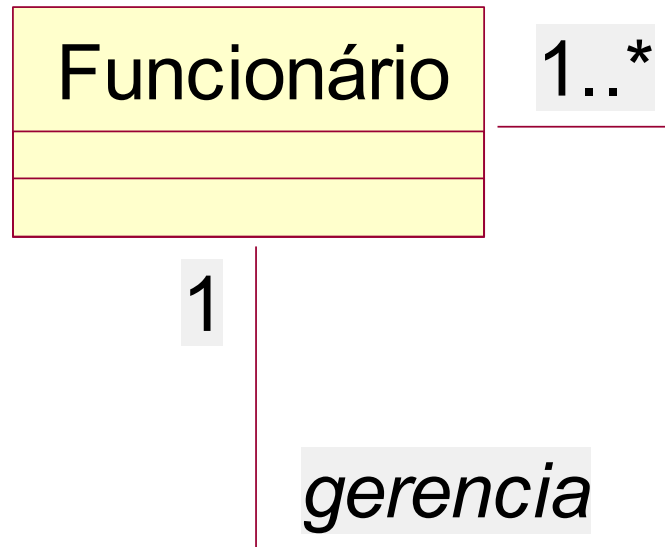
- A associação é um relacionamento estrutural que especifica que objetos de um elemento estão conectados a objetos de outro elemento



- É a cardinalidade de uma associação

1	Classe	exatamente 1
*	Classe	muitos (zero ou mais)
0.. 1	Classe	opcional (zero ou um)
m.. n	Classe	seqüência especificada

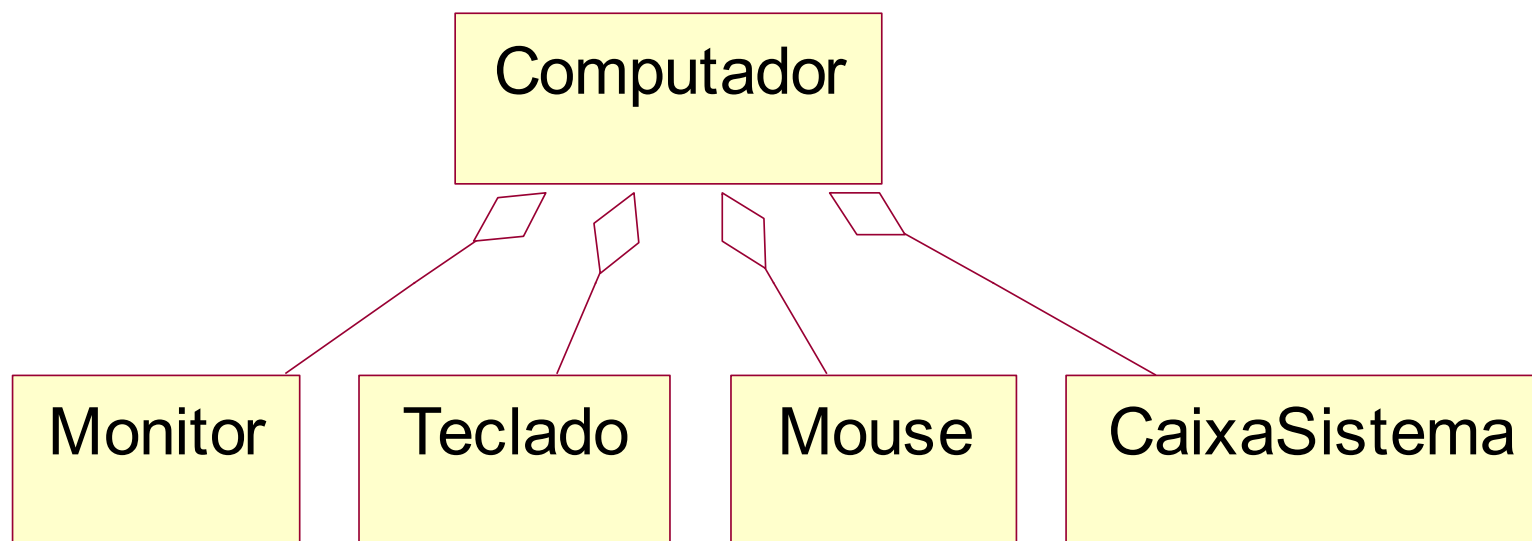
- Quando há um relacionamento de uma classe para consigo própria



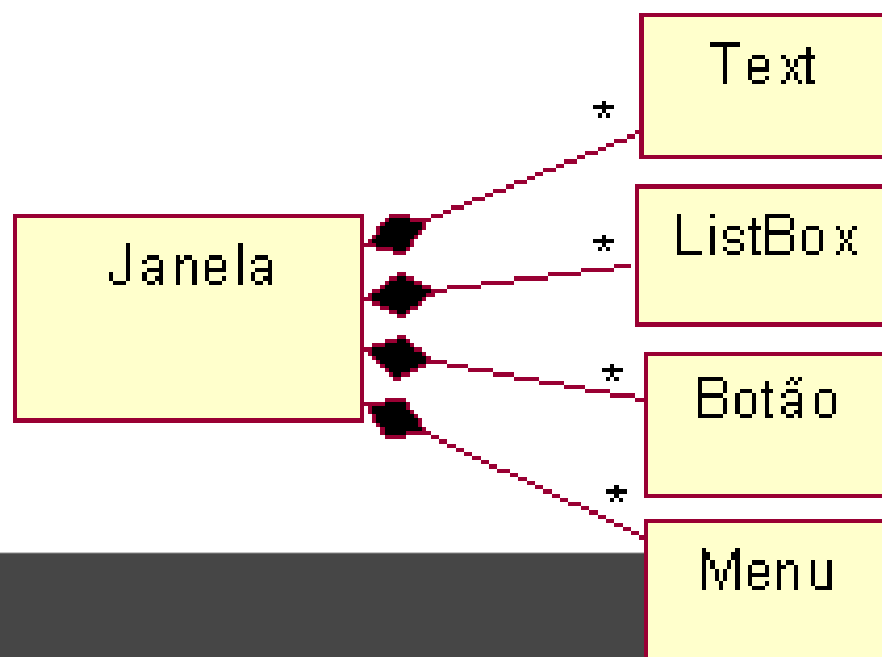
- Quando há duas classes envolvidas na associação de forma direta de uma para a outra



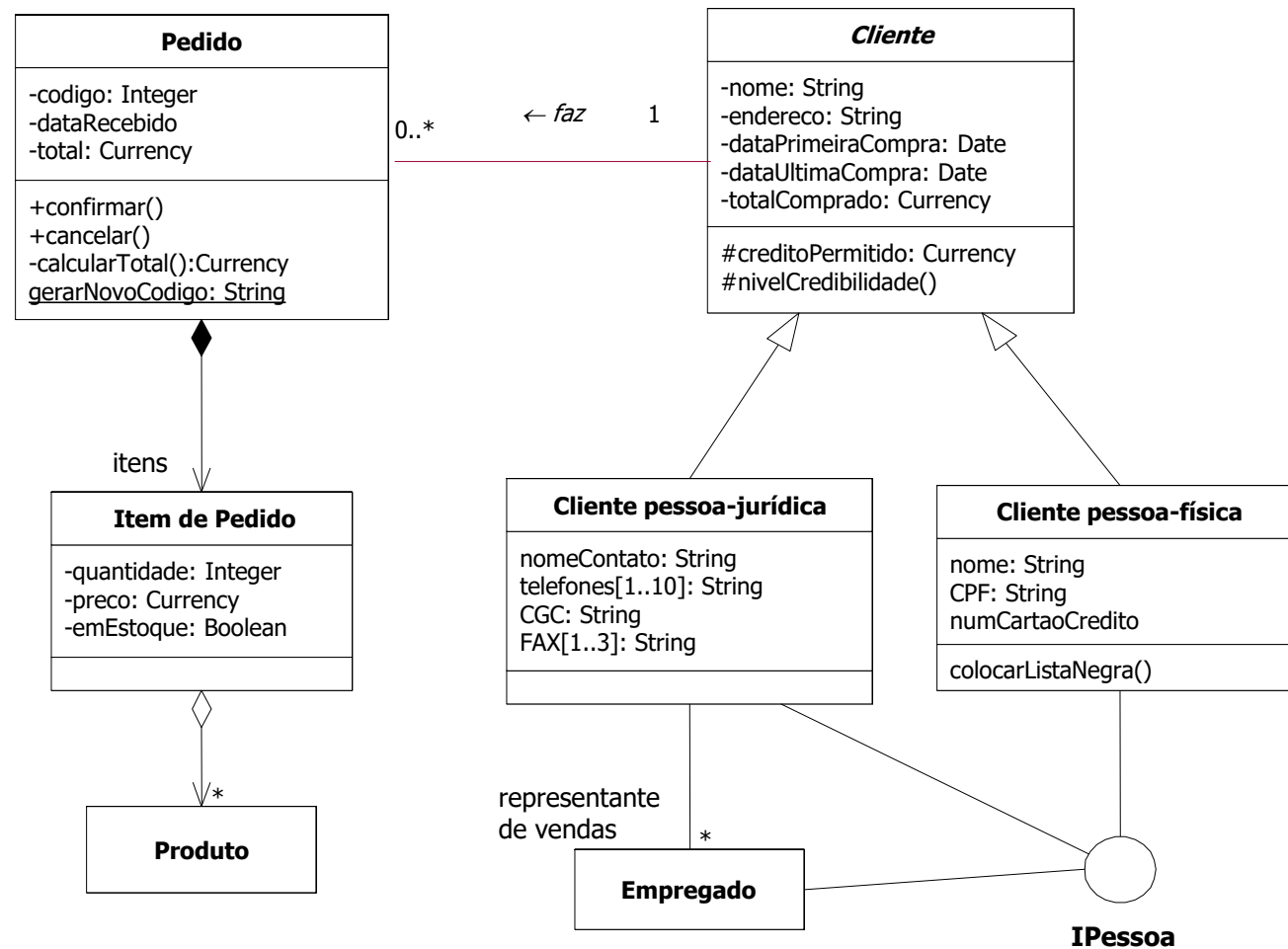
- Uma forma especial de associação entre o todo e suas partes, no qual o todo é composto de partes
- Não impõe que a vida das “Partes” esteja relacionado com a vida do “Todo”



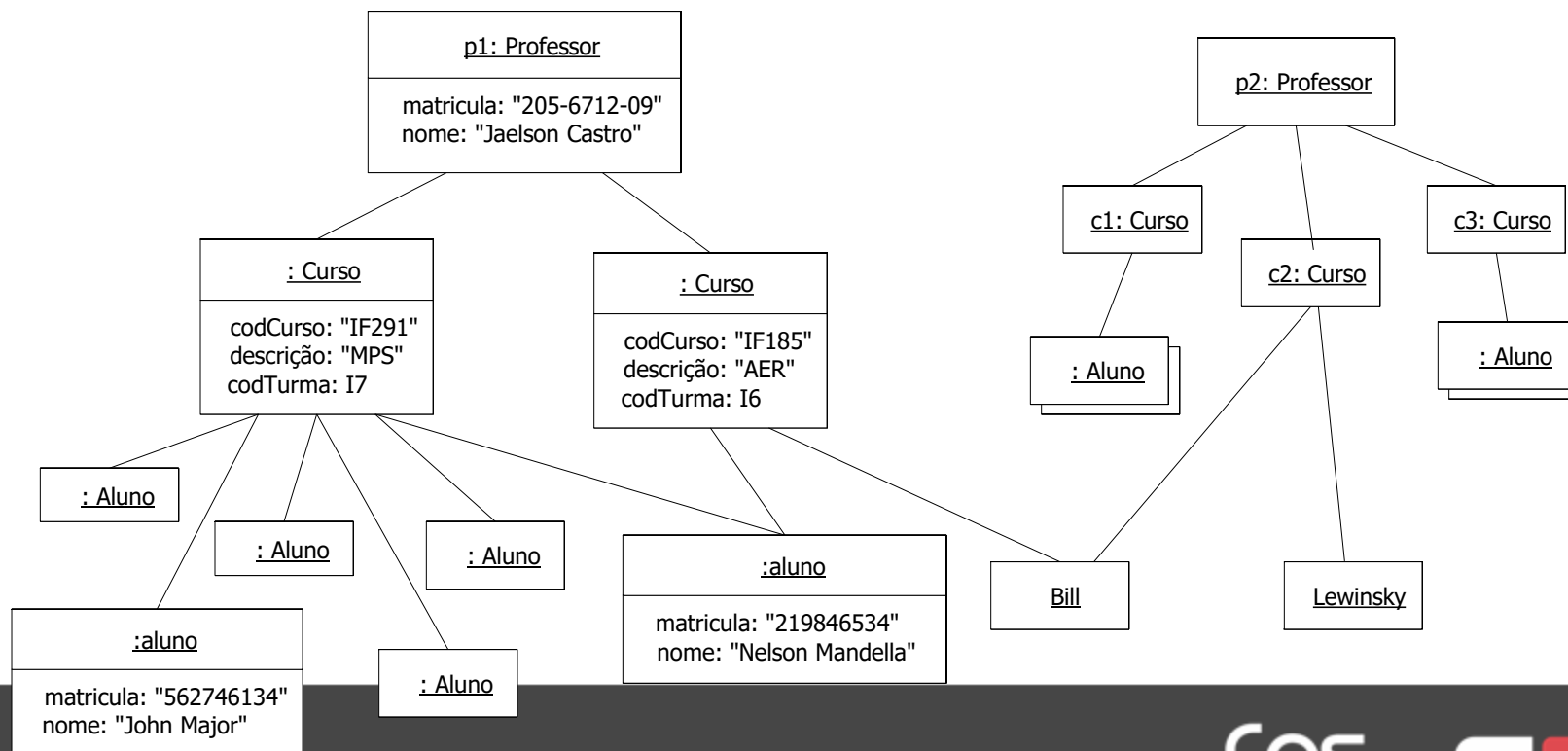
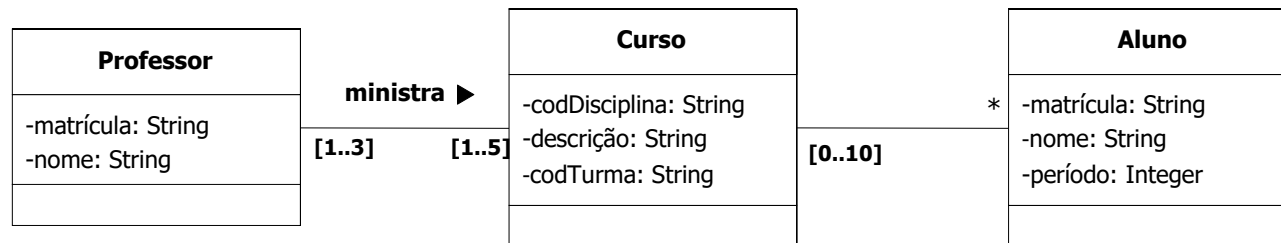
- Uma forma mais forte de agregação
- Há uma coincidência da vida das partes
- Uma vez criada a parte ela irá viver e morrer com ele
- O “Todo” é responsável pelo gerenciamento da criação e destruição das partes



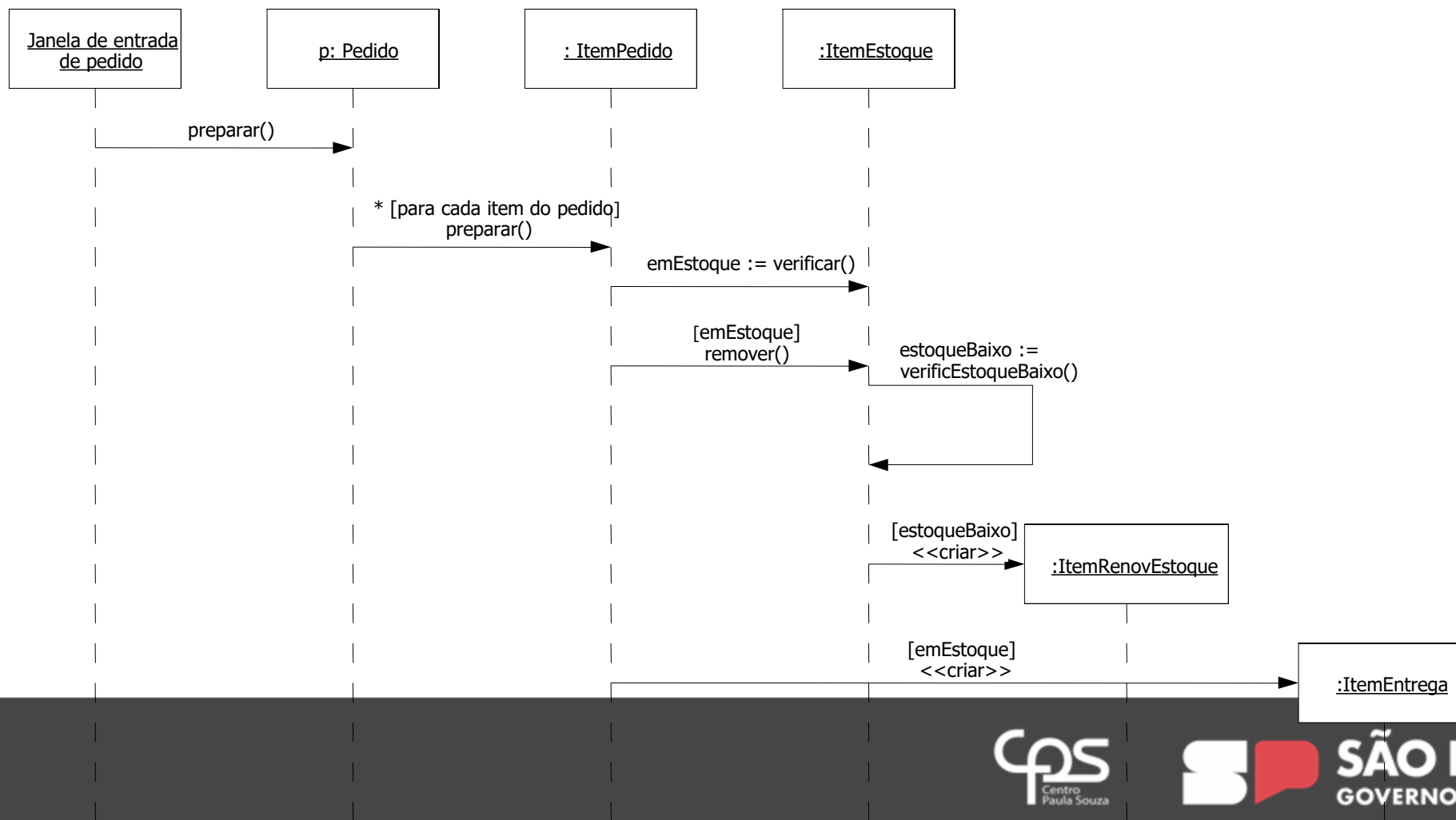
- Os diagramas de classes são os principais diagramas estruturais da UML
- Diagramas de classe mostram classes, interfaces e seus relacionamentos
- As classes especificam a estrutura e o comportamento dos objetos, que são instâncias de classes.



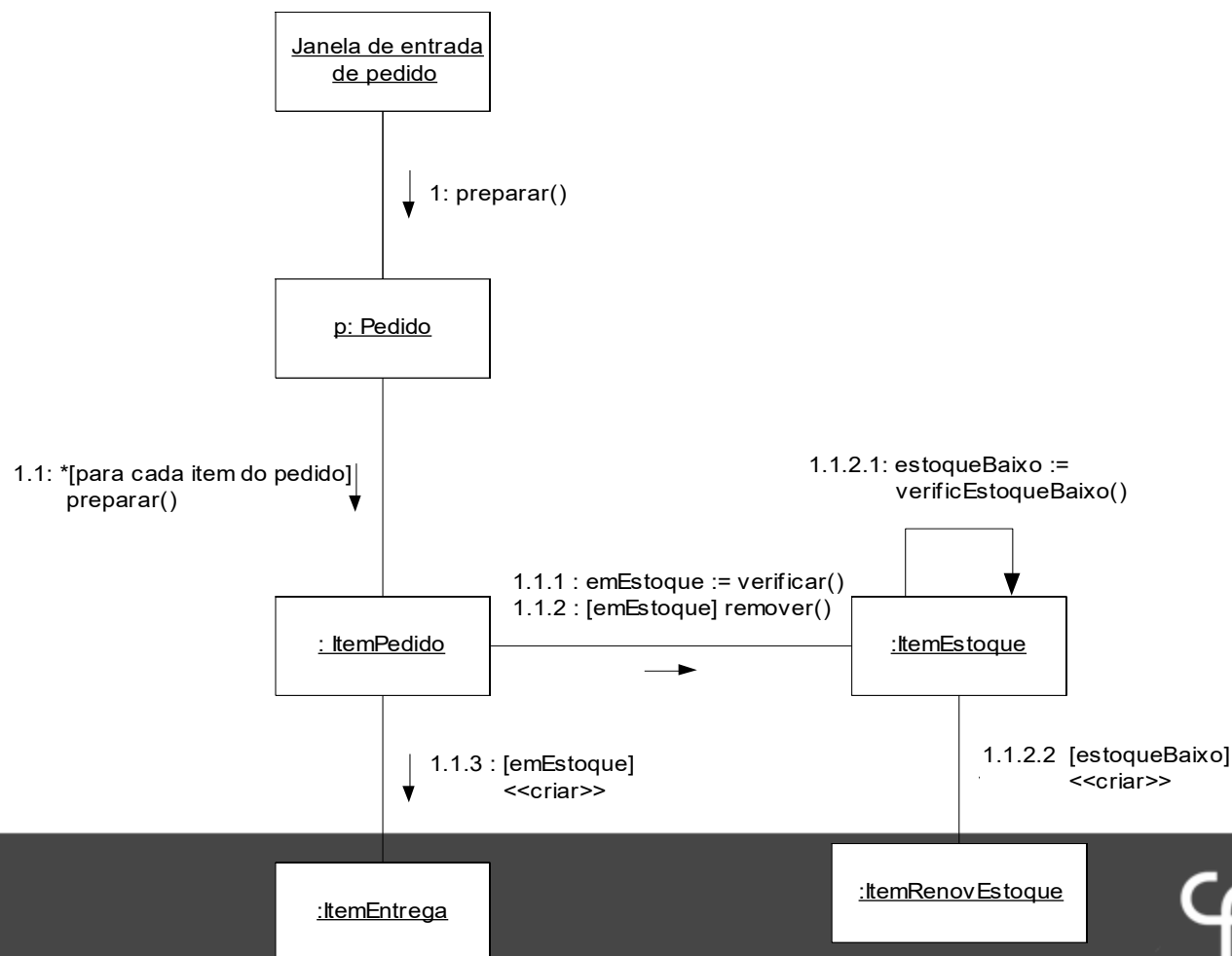
- Mostram objetos e seus relacionamentos
- Representam instâncias estáticas de elementos dos diagramas de classes
- Os diagramas de objetos são úteis para a modelagem de estruturas de dados complexas



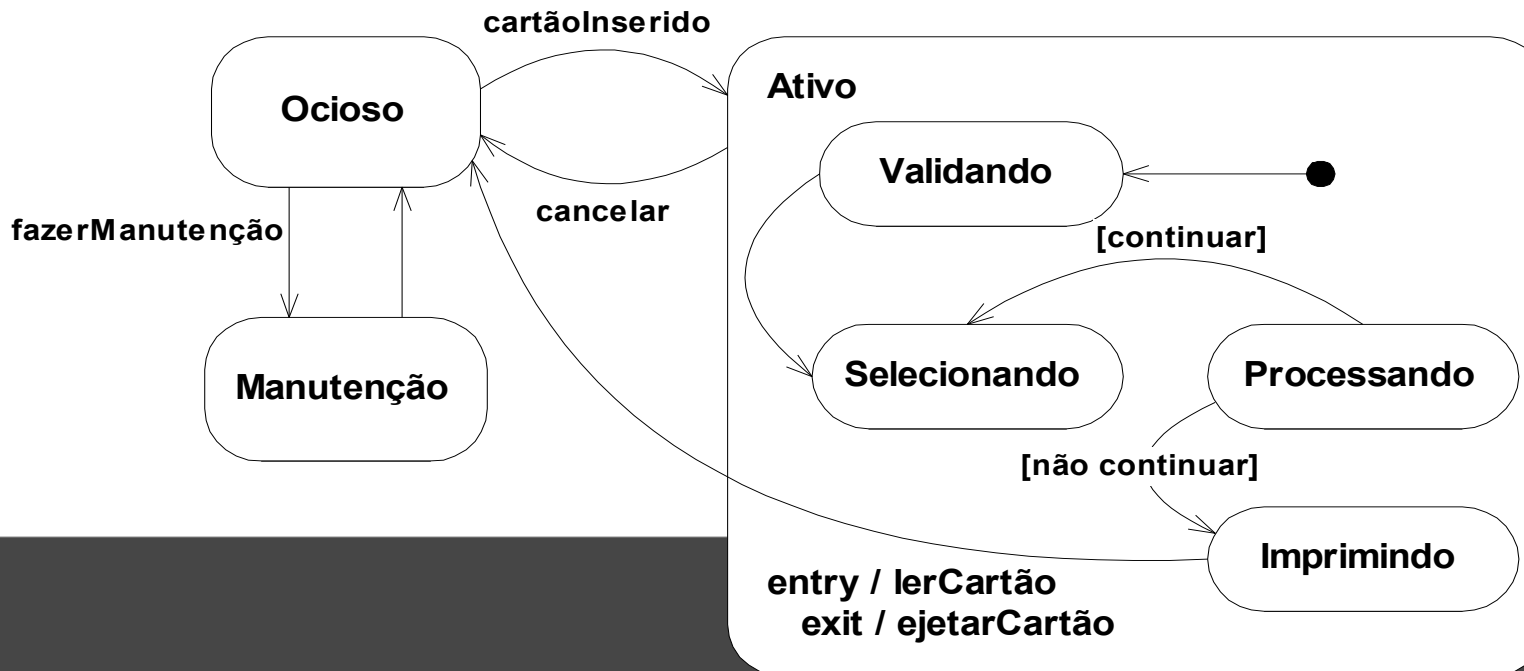
- Mostra um conjunto de objetos, seus relacionamentos e as mensagens que podem ser enviadas entre eles



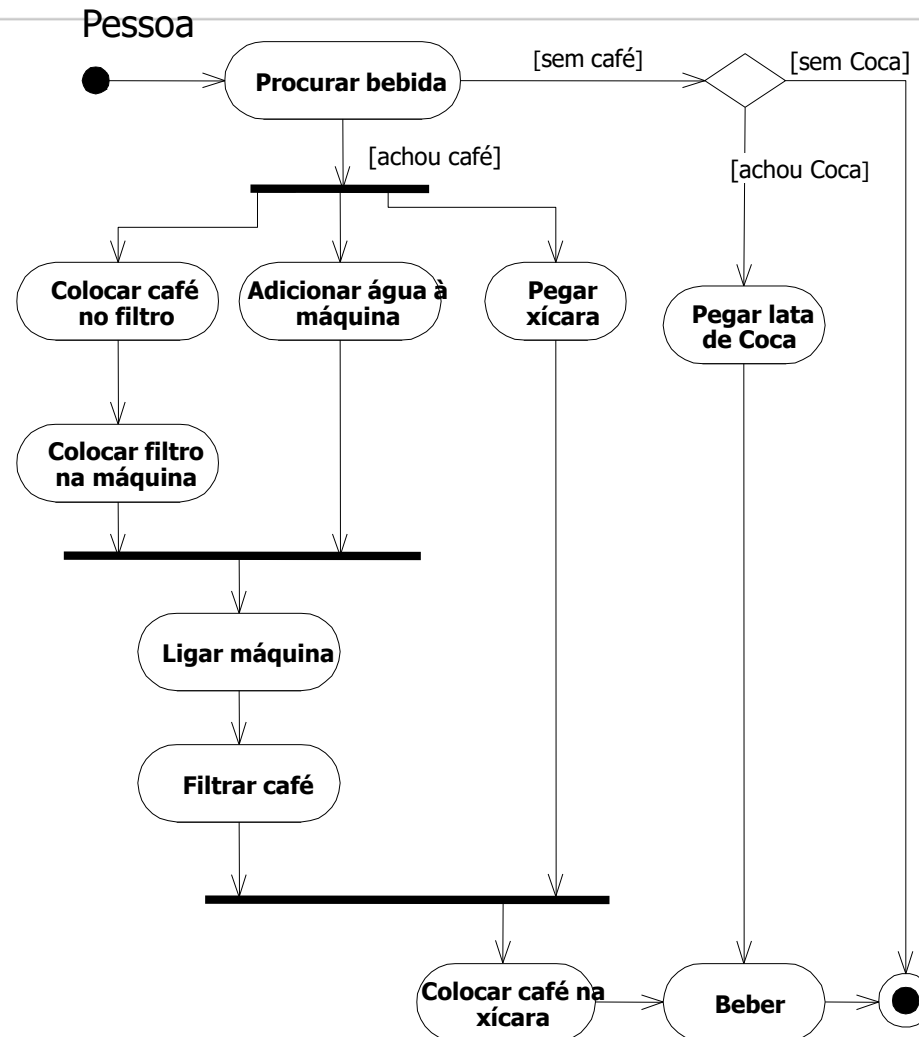
- Mostra um conjunto de objetos, seus relacionamentos e as mensagens que enfatizam a organização dos objetos que trocam mensagens



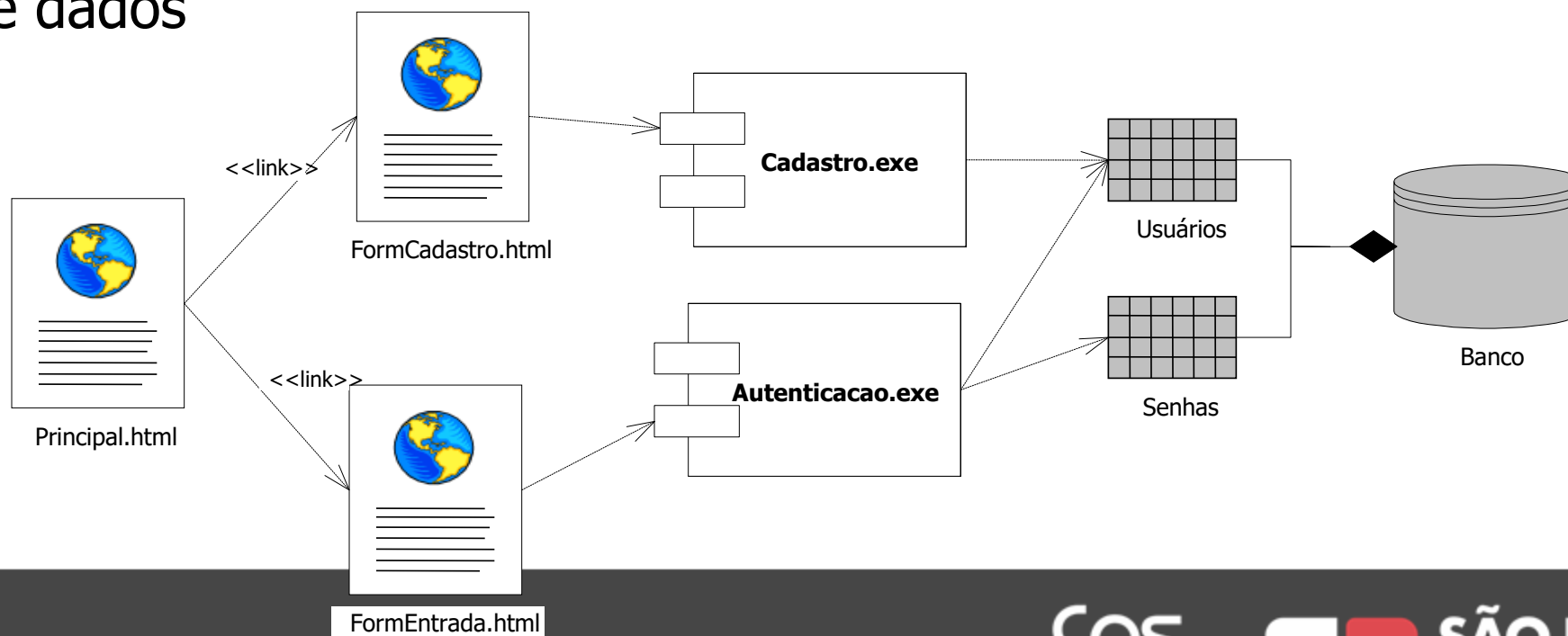
- Mostra uma máquina contendo estados, transições, eventos e atividades
- Estes diagramas são usados para modelar o comportamento de objetos (com comportamento complexo)
- Nestes diagramas são modelados os estados em que um objeto pode estar e os eventos que fazem o objeto passar de um estado para outro



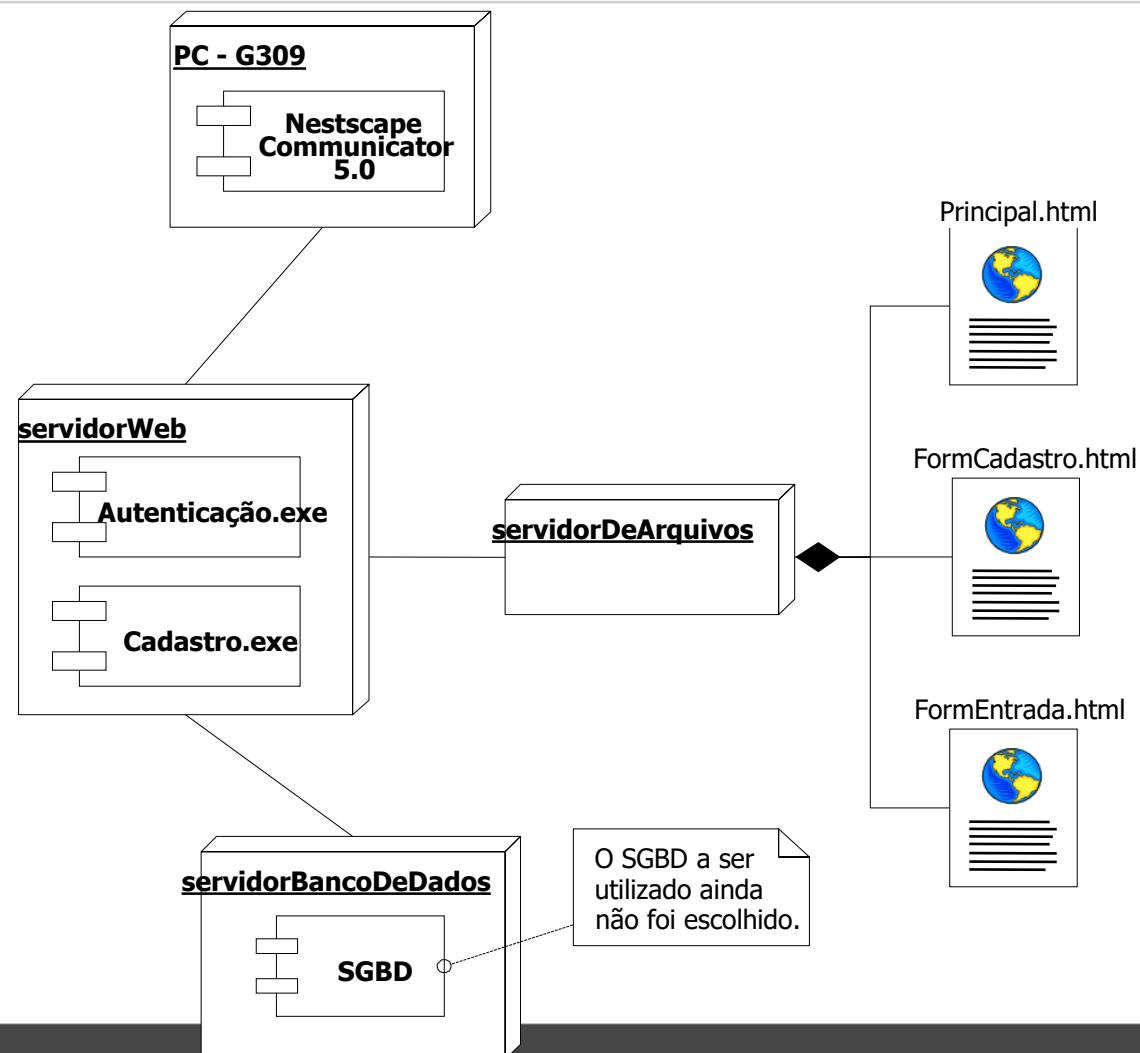
- Destaca a lógica de realização de uma tarefa
- Mostra o fluxo entre atividades (ações não-atômicas)
- É semelhante aos antigos fluxogramas
- É usado também para modelar alternativas de execução e atividades concorrentes



- Mostra os componentes de hardware e software de uma aplicação e os relacionamentos entre eles
- É usado para modelar o aspecto físico de um sistema
- Exemplos de componentes são documentos, executáveis e tabelas de bancos de dados



- É usado para modelar a arquitetura de distribuição em que o sistema será executado
- É composto por nós e relacionamentos de comunicação
- Um nó pode ser um computador, uma rede, um disco rígido, um sensor, etc.





Vendas.drawio

Arquivo Editar Exibir Ordenar Extras Ajuda Última alteração há 2 minutos

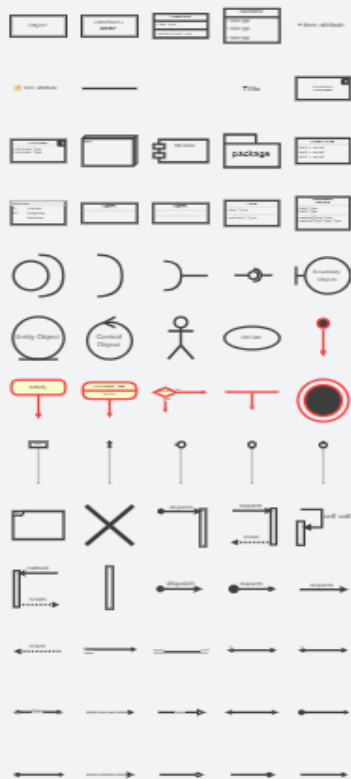


Compartilhar

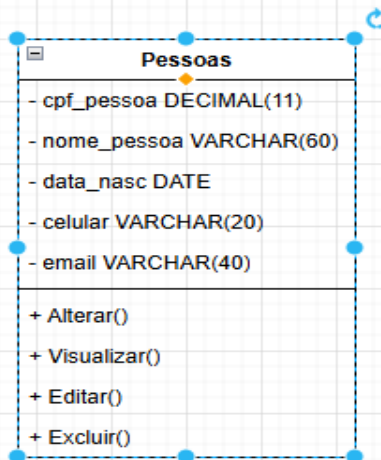


Relação de entidade

UML



+ Mais formas



Estilo

Texto

Ordenar



Preenchimento

- ☒ Preenchimento Auto
- ☐ Gradiente
- ☐ Cor da raia

Linha

- ☒ Linha
- 1 pt
- Perímetro 0 pt
- Opacidade 100 %

Efeitos

- Editar
- Copiar estilo
- Definir como estilo padrão

Propriedade Valor

